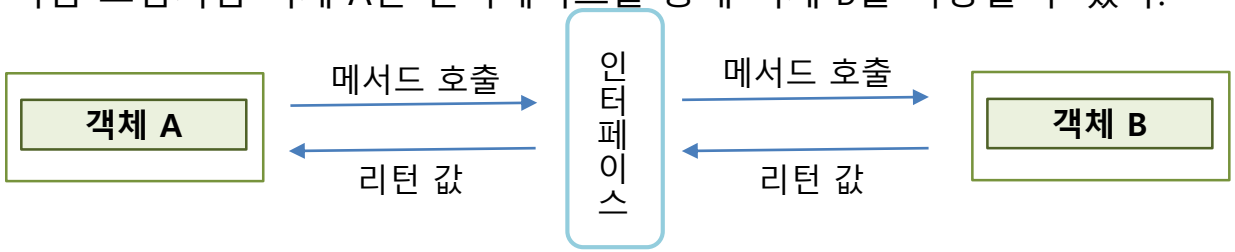
16강. 인터페이스

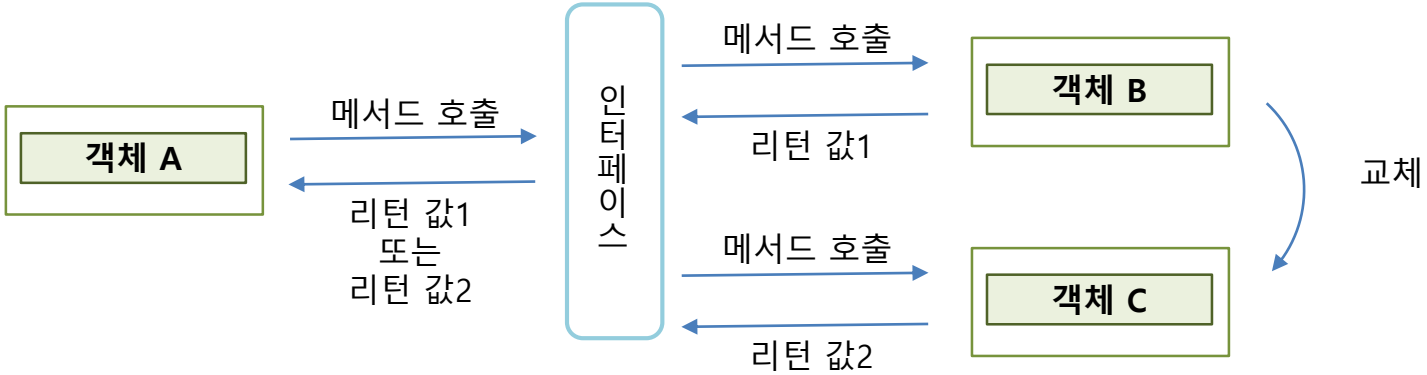
- 인터페이스의 이해
- 인터페이스의 문법
- 다형성
- 인터페이스 정리

16-1. 인터페이스의 이해

- 인터페이스(interface)는 사전적 의미로 두 장치를 연결하는 접속기를 말한다. 두 장치를 서로 다른 객체로 본다면 인터페이스는 이 두 객체를 연결하는 역할을 한다.
- 다음 그림처럼 객체 A는 인터페이스를 통해 객체 B를 사용할 수 있다.



- 객체 A가 인터페이스의 메서드를 호출하면, 인터페이스는 객체 B의 메서드를 호출하고 그 결과를 받아 객체 A로 전달해준다. 객체 A가 객체 B의 메서드를 직접 호출하면 간단한데 왜 중간에 인터페이스를 거치는 걸까?
- 다음 그림과 같이 객체 B가 객체 C로 교체된다고 가정해보자



객체 A는 인터페이스의 메서드만 사용하므로 객체 B가 객체 C로 변경된 것에는 관심이 없다. 만약 직접 호출하는 구조라면 객체 B에서 객체 C로 변경하는 수정작업이 필요할 것이다.
객체 A가 인터페이스의 메서드를 호출하면 객체 B 또는 객체 C의 메서드이다. 만약 객체 B의 메서드 실행 결과와 객체 C의 메서드 실행 결과가 다르면 객체 교체로 인해 다른 결과를 얻게 된다. 이 특징으로 인터페이스는 다형성 구현의 주된 기술로 이용된다.

16-2. 인터페이스의 문법

인터페이스 선언

- 인터페이스(interface) 선언은 class 키워드 대신에 **interface 키워드**를 사용한다.
- 접근제한자는 **default(동일 package), public(전체)을 사용**할 수 있다.

```
interface 인터페이스명 { ... }          // default 접근 제한
public interface 인터페이스명 { ... }   // public 접근 제한
```

16-2. 인터페이스의 문법

인터페이스 선언

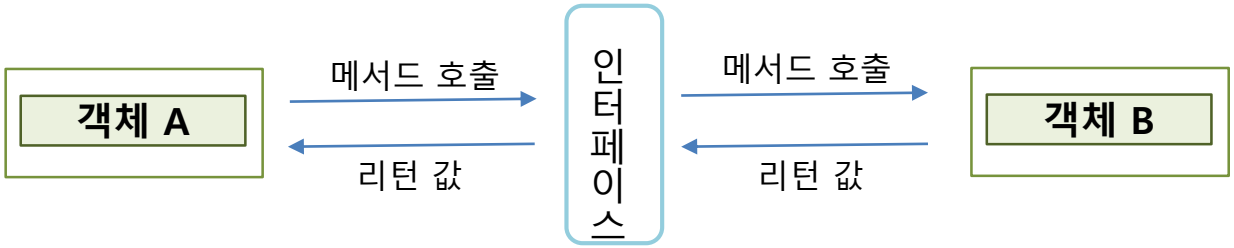
< 중괄호 안에는 인터페이스가 가지는 멤버들 >

```
interface Animal {
    // 1. 상수
    int A = 100; // → public static final

    // 2. 추상 메서드
    void sound(); // → public abstract

    // 3. default 메서드 (Java 8)  → Animal a = new Dog(); a.move(); 객체로 호출
    default void move() {
        System.out.println("move");
    }

    // 4. static 메서드 (Java 8) → Animal.info() 호출
    static void info() {
        helper();          // 내부 호출
    }

    // 5. private 메서드 (Java 9+) → 인터페이스 내부 호출만 가능한 메서드
    private void helper() {
        System.out.println( " 공통 로직");
    }
}
```

16-2. 인터페이스의 문법

구현 클래스 선언

- 객체 A가 인터페이스의 추상 메서드를 호출하면 인터페이스는 객체 B의 메서드를 실행한다. 이것은, 객체 B가 인터페이스에 선언된 추상 메서드와 동일한 선언부를 가진 메서드를 가지고 있어야 한다.



- 여기서 **객체 B를 인터페이스를 구현한(implement) 객체**라고 한다.
- 객체 B와 같은 **구현 객체**는 **인터페이스를 구현하고 있다고 선언부에 명시**해야 한다.

```
public class B implements 인터페이스명 { ... }
```

- implements 키워드는 해당 클래스가 인터페이스를 통해 사용할 수 있다는 표시
- 인터페이스의 추상 메서드를 재 정의한 메서드가 있다는 뜻이다.

16-2. 인터페이스의 문법

변수 선언과 구현 객체 대입

- 인터페이스도 하나의 타입이며, 변수의 타입으로 사용할 수 있다.
- 인터페이스는 **참조 타입**으로 인터페이스 변수에 객체를 참조하고 있지 않다는 의미로 **null 대입도 가능**하다.

```
RemoteControl rc;
RemoteControl rc = null;
```

- **인터페이스를 통한 구현 객체 사용**은, 인터페이스 변수에 **구현 객체를 대입**한다. 이는 구현 객체의 번지를 대입하는 것이다.

< Television 객체 생성 후 번지 대입 코드 >

```
rc = new Televisioin;
```

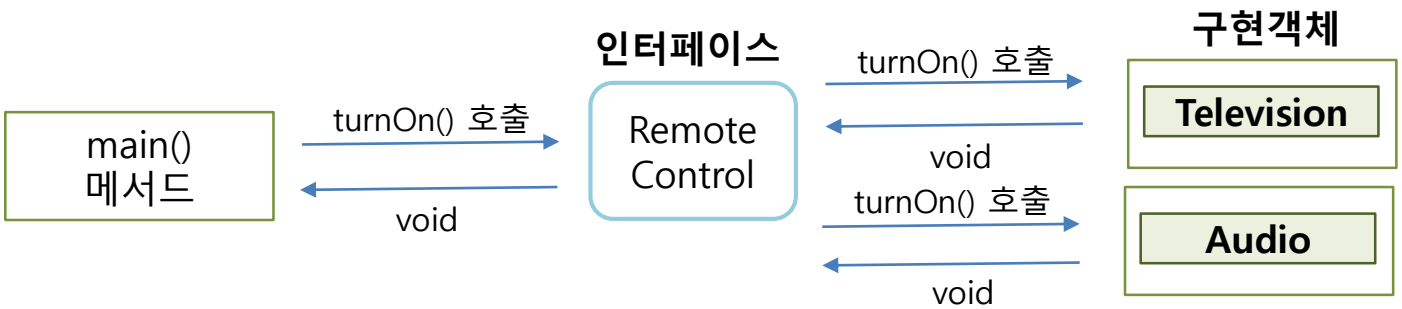
- 인터페이스 변수에 구현 객체가 대입되었다면, 변수를 통해 인터페이스의 추상 메서드를 호출할 수 있다.

```
rc.turnOn();
```

16-2. 인터페이스의 문법

인터페이스 예제(java_16_2_ex1_interface)

- ❖ RemoteControl 인터페이스를 정의한다.
- ❖ 구현 객체 Television, Audio 만들고, "TV를 켭니다", "TV를 끕니다", "Audio를 켭니다.", "Audio를 끕니다." 를 출력하는 메서드를 가진 예제 프로그램을 만드시오.



16-2. 인터페이스의 문법

상수 필드

- 인터페이스는 **public static final 특성**을 갖는 **상수 필드를 멤버**로 가질 수 있다.

[public static final] 타입 상수명 = 값;

- 인터페이스에 선언된 모든 필드는 **public static final 특성**을 갖기 때문에, public static final을 생략하더라도 컴파일 과정에서 자동으로 붙게 된다.
- **상수 명은 대문자로 작성**하고, 단어 연결이 필요하면 언더바(_)로 연결하는 것이 **관례**이다.

(interface 예제: RemoteControl.java)

```
package kr.co.javaex.interface;

public interface RemoteControl {
  // 상수 선언
  int MAX_VOLUMN = 10;
  int MIN_VOLUMN = 0;
}
```

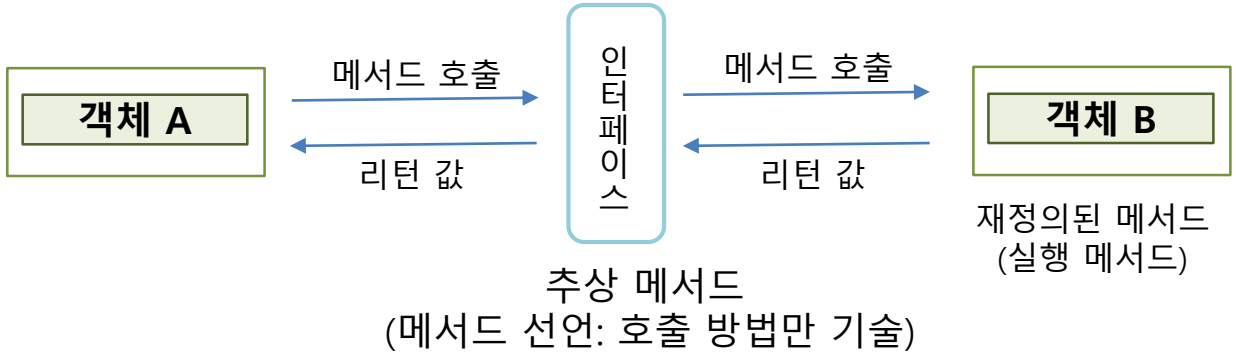
16-2. 인터페이스의 문법

추상 메서드

- 인터페이스는 구현 클래스가 재정의해야 하는 public 추상 메서드(abstract method)를 멤버로 가질 수 있다.
- 선언형식은 아래와 같고, public abstract를 생략하더라도 컴파일 과정에서 자동으로 붙는다.

```
[ public abstract ] 리턴타입 메서드명( 매개변수, ... );
```

- 인터페이스의 추상 메서드는 객체 A가 인터페이스를 통해 메서드를 호출할 수 있는 방법을 알려주는 역할을 한다.
- 인터페이스 구현 객체 B는 추상 메서드가 재정의된 실행 메서드가 있어야 한다.



16-2. 인터페이스의 문법

인터페이스 예제(java_16_2_ex2_interface)

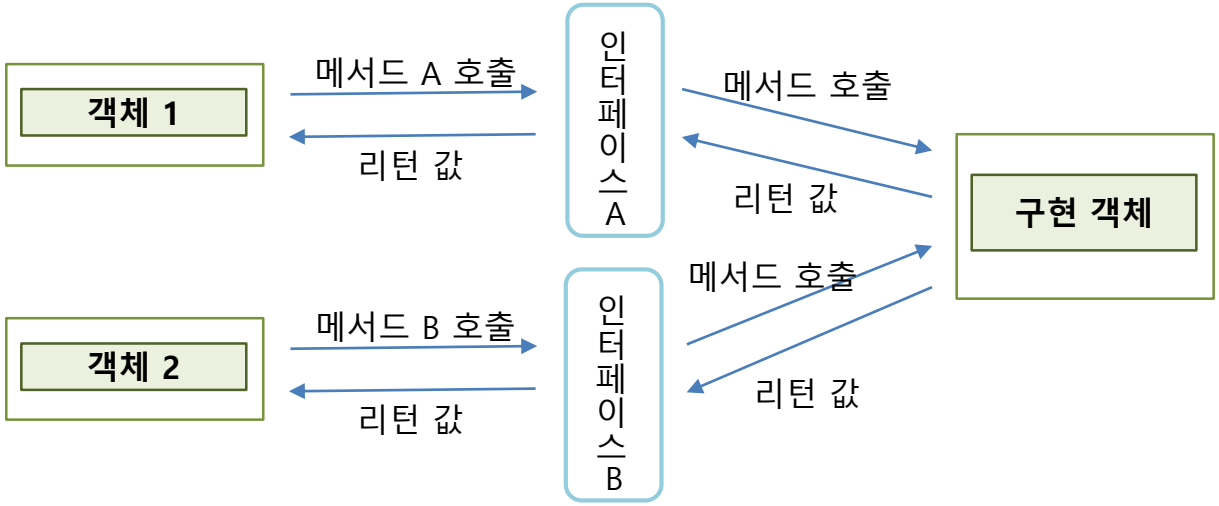
- 앞에서 선언한 RemoteControl 인터페이스에 상수 필드 MAX_VOLUMN, MIN_VOLUMN와 추상 메서드turnOn(), turnoff(), setVolumn(int volumn) 를 선언한다.
- RemoteControl 인터페이스를 구현한 Television, Audio 객체를 만든다.
- 볼륨은 MIN_VOLUMN(0)과 MAX_VOLUMN(10) 범위를 벗어날 수 없다(제약 조건 구현).
- 아래의 실행결과를 출력하는 예제 프로그램을 만드세요.
  - "TV를 켭니다."
  - "현재 TV 볼륨: 5(입력 값 예, 음수-> 0, 10 이상 -> 10)"
  - "TV를 끕니다."

  - "Audio를 켭니다."
  - "현재 Audio 볼륨: 5(입력 값 예)"
  - "Audio를 끕니다."

16-2. 인터페이스의 문법

다중 인터페이스 구현

➢ 구현 객체는 여러 인터페이스를 implements 할 수 있다. 즉, 구현 객체가 인터페이스 A와 인터페이스 B를 구현하고 있다면 각각 인터페이스를 통해 구현 객체를 사용할 수 있다.



< 다중 인터페이스 구현 클래스 선언 형식 >

```
public class 구현 클래스명 implements 인터페이스A, 인터페이스B, ... {
  // 모든 추상 메서드 재정의
}
```

➢ 인터페이스A와 인터페이스B를 구현한 객체는 아래와 같이 두 인터페이스 타입 변수에 각각 대입될 수 있다. 구현 객체가 어떤 인터페이스 변수에 대입되는지에 따라 변수가 호출할 수 있는 추상 메서드가 결정된다.

```
인터페이스A 변수 = new 구현클래스명( ... );
인터페이스B 변수 = new 구현클래스명( ... );
```

16-2. 인터페이스의 문법

다중 인터페이스 구현 예제(java_16_2_ex3_interface)

❖ 아래와 같이 RemoteControl 인터페이스와 Searchable 인터페이스 둘 다를 구현한 SmartTelevision 클래스를 가지는 예제 프로그램을 만드시오.

(RemoteControl.java)

```
package kr.co.javaex.interface;

public interface RemoteControl {
  // 추상 메서드
  void trunOn();
  void turnoff();
}
```

(Searchable.java)

```
package kr.co.javaex.interface;

public interface Searchable {
  // 추상 메서드
  void search(String url);
}
```

16-2. 인터페이스의 문법

인터페이스 상속

- 인터페이스도 다른 인터페이스를 상속할 수 있으며, 클래스와 달리 **다중 상속을 허용**한다.

< 사용 형식 >

```
public interface 자식 인터페이스명 extends 부모인터페이스1, 부모인터페이스2 { ... }
```

- **자식 인터페이스 구현 클래스**는 자식 인터페이스 메서드와 부모 인터페이스의 **모든 추상 메서드를 재정의**해야 한다.
- **구현 객체**는 **자식 및 부모 인터페이스 변수에 대입**될 수 있다.

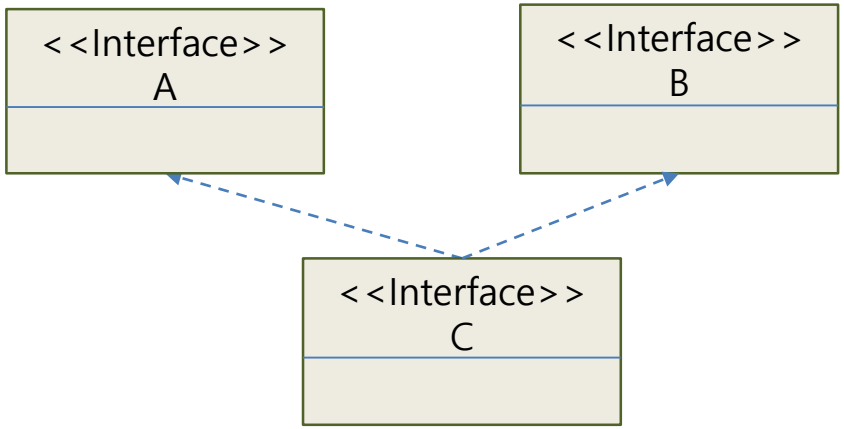
```
자식인터페이스 변수 = new 구현클래스( ... );
부모인터페이스1 변수 = new 구현클래스( ... );
부모인터페이스2 변수 = new 구현클래스( ... );
```

- **구현 객체가 자식 인터페이스 변수에 대입**되면, **자식 및 부모 인터페이스의 모든 추상 메서드를 호출**할 수 있다.
- **부모 인터페이스 변수에 대입**되면 **부모 인터페이스에 선언된 추상 메서드만 호출** 할 수 있다.

16-2. 인터페이스의 문법

인터페이스 예제(java_16_2_ex4_interface)

- ❖ interface A, interface B, interface C(interface A와 interface B를 상속)
- ❖ class InterfaceCImpl(interface C 구현 객체)
- ❖ 아래의 실행 결과를 출력하는 예제 프로그램을 만드시오.




```
InterfaceCImpl-methodA()  실행

InterfaceCImpl-methodB()  실행

InterfaceCImpl-methodA()  실행
InterfaceCImpl-methodB()  실행
InterfaceCImpl-methodC()  실행
```

16-2. 인터페이스의 문법

자동 타입 변환

- 인터페이스의 타입 변환은 **인터페이스와 구현 클래스 간**에 발생한다.
- 인터페이스 변수에 구현 객체를 대입하면 **구현 객체는 인터페이스 타입으로 자동 타입 변환**된다.
- 자동 타입 변환(promotion)은 자동으로 타입 변환이 일어나는 것을 말한다.

자동타입변환

```
인터페이스 변수   =   구현객체;
```

- 구현 객체가 인터페이스 타입으로 **자동 타입변환**되면, **인터페이스에 선언된 메서드만 사용** 가능하다.
- **부모 클래스가 인터페이스를 구현**하고 있다면 **상속 받은 자식 클래스도 인터페이스 타입으로 자동 타입 변환**될 수 있다.

강제 타입 변환

- **캐스팅(casting) 기호를 사용**해서 인터페이스 타입을 **구현 클래스 타입으로 변환**시키는 것을 말한다.

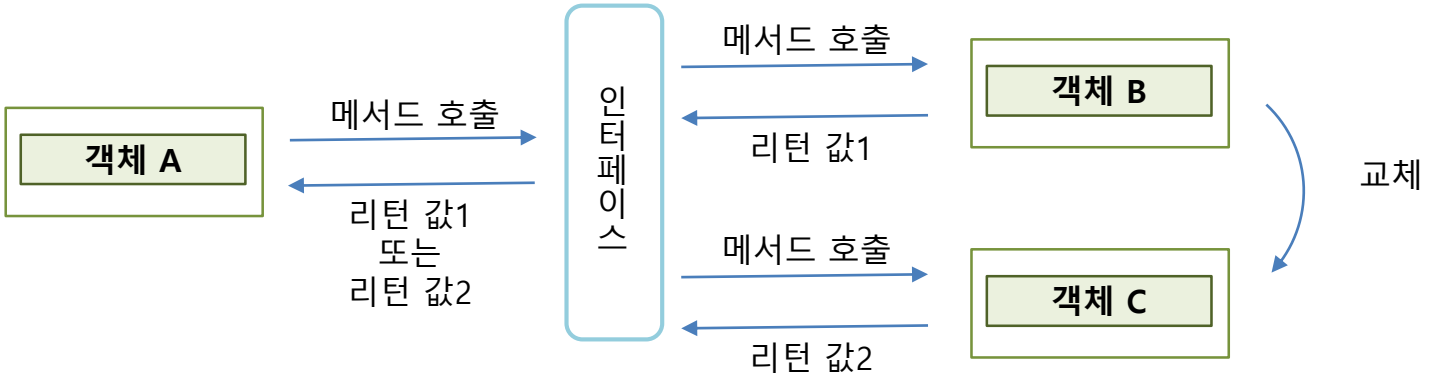
강제타입변환

```
구현클래스 변수  = (구현클래스) 인터페이스변수;
```

- 자동 타입 변환 후에 구현 객체의 메서드를 사용하고 싶다면 캐스팅 기호를 사용하여 인터페이스 변수를 강제 타입 변환해야 한다.

16-3. 다형성

- 다형성이란 사용 방법은 동일하지만 실행 결과가 다양하게 나오는 성질을 말한다.
- 아래 그림에서 구현 객체 B와 구현 객체 C 중 어떤 객체가 인터페이스에 대입 되었는가에 에 따라 객체 A의 메서드 호출 결과가 달라질 수 있다.



- 인터페이스의 다형성을 구현하기 위해서는 자동 타입 변환과 메서드 재정의가 필요하다.

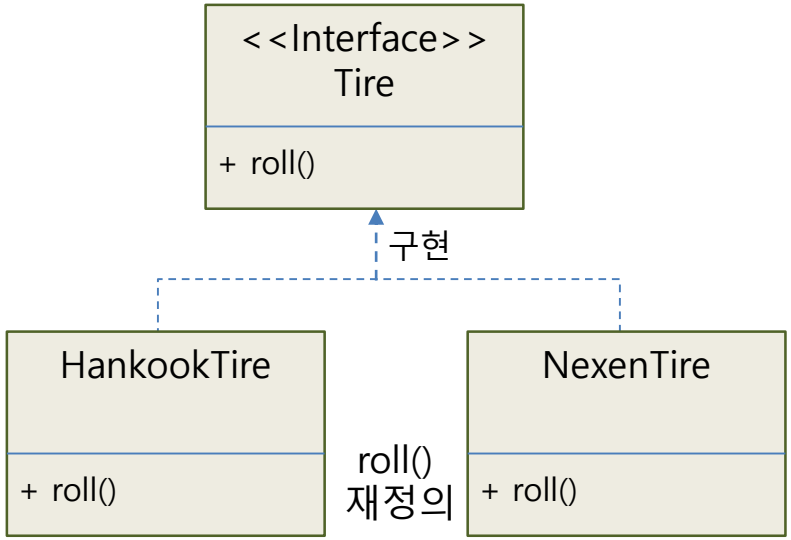
자동 타입 변환 + 메서드 오버라이딩 → 다형성

- 인터페이스의 추상 메서드는 구현클래스에서 재정의 해야 하며, 재정의 내용은 구현 클래스 마다 다르다.
- 구현 객체는 인터페이스 타입으로 자동 타입 변환되고, 인터페이스 메서드 호출 시 구현 객체의 재정의된 메서드가 호출되어 다양한 실행 결과를 얻을 수 있다.

16-3. 다형성

- **필드의 다형성**

➢ **필드 타입은 동일**하지만(사용 방법은 동일), **대입되는 객체가 달라져서 실행 결과가 다양**하게 나올 수 있는 것을 말한다.




```
Interface Tire {
    // 추상 메서드
    void roll();
}
```

```
public class HankookTire implements Tire {
    // 추상 메서드 재정의
    @Override
    public void roll() {
        System.out.println("한국타이어가 구른다");
    }
}
```

```
public class Car {
  // 필드
  Tire tire = new HankookTire();

  // 메서드 선언
  void run() {
    tire.roll()          // tire 필드에 대입된 객체의 roll() 메서드 호출
  }
}
```

➢ 실행 메서드에서 Car를 호출 후 객체를 NexenTire로 변경 후 메서드를 호출함으로써 실행 결과가 달라지는 다형성을 표현한다.

16-3. 다형성

- **필드의 다형성**

```
Car myCar = new Car();              // Car  객체 생성

// HankookTire 장착
myCar.tire = new HankookTire();  // 자동 타입 변환

// NexenTire 장착
myCar.tire = new NexenTire();      // 자동 타입 변환
```

- Car 클래스의 run() 메서드는 tire 필드에 대입된 인터페이스의 roll() 메서드를 호출한다.
- **구현 객체인 HankookTire와 NexenTire가 roll() 메서드를 재정의**하고 있다면 **재정의된 roll() 메서드가 호출**된다.

```
myCar.run();       -> 대입된 tire의 roll() 메서드가 호출
```

- 장착한 tire객체에 따라 roll() 메서드의 실행 결과가 달라지게 된다. 이것이 **필드의 다형성**이다.

16-3. 다형성

- **필드의 다형성 – 사용 예**

```java
public interface Tire {
    // 추상 메서드
    public void roll();
}
```

```java
public class HankookTire implements Tire {
    // 추상메서드 재정의(오버라이딩)
    @Override
    public void roll() {
        System.out.println("한국타이어가 회전합니다.");
    }
}
```

```java
public class NexenTire implements Tire {
    // 추상메서드 재정의(오버라이딩)
    @Override
    public void roll() {
        System.out.println("넥센타이어가 회전합니다.");
    }
}
```

```java
public class Car {
    // 필드 선언
    public Tire tire;

    // 메서드 선언
    public void run() { // 인터페이스에 선언된 추상메서드 호출
        tire.roll();    // 대입된 구현 객체의 roll() 메서드 호출
    }
}
```

```java
public static void main(String[] args) {
    // Car 객체 생성
    Car myCar = new Car();

    // 한국타이어 객체 대입
    myCar.tire = new HankookTire();
    myCar.run();

    // 넥센타이어 객체 대입
    myCar.tire = new NexenTire();
    myCar.run();
}
```

16-3. 다형성

- **매개변수의 다형성**

➢ 메서드 호출 시 매개 값을 다양화하기 위해 매개변수 타입을 인터페이스로 선언을 하면 다양한 구현 객체를 대입할 수 있다.

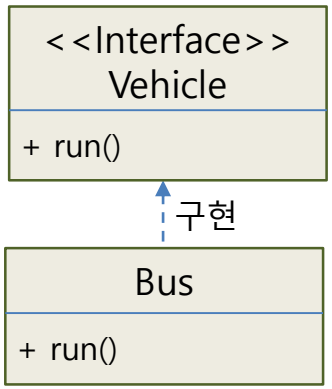
다음과 같이 Driver 클래스가 있고 Vehicle 매개 변수를 갖는 driver() 메서드가 정의되어 있다고 가정해보자

```
public class Driver {
  public void drive(Vehicle vehicle) {
    vehicle.run();
  }
}
```

구현 객체가 대입될 수 있도록 매개변수를 인터페이스타입으로 선언

인터페이스 추상 메서드 호출

- Bus가 Vehicle의 구현 클래스이면 Driver의 drive() 메서드를 호출할 때 Bus 객체를 생성, 매개 값으로 줄 수 있다.
- drive()메서드를 호출할 때 인터페이스 Vehicle을 구현하는 어떠한 객체라도 매개 값으로 줄 수 있는데, 어떤 객체를 주느냐에 따라 run() 메서드의 실행 결과가 다르게 나온다. 매개변수의 다형성이다.



메서드 재정의

```
Driver driver = new Driver();
Bus bus = new Bus();

driver.drive(bus);
```

자동 타입 변환 발생
Vehicle vehicle = bus;

16-3. 다형성

- **매개변수의 다형성 – 사용 예**

```java
public interface Vehicle {
    // 추상메서드 선언
    void run();
}
```

```java
public class Bus implements Vehicle {
    // 추상메서드 재정의(오버라이딩)
    @Override
    public void run() {
        System.out.println("버스가 달립니다.");
    }
}
public class Taxi implements Vehicle {
    // 추상메서드 재정의(오버라이딩)
    @Override
    public void run() {
        System.out.println("택시가 달립니다.");
    }
}
```

```java
public class Driver {
    // 구현 객체 대입 가능토록 인터페이스 타입으로 선언
    public void driver(Vehicle vehicle) {
        vehicle.run();  // 인터페이스 메서드 호출
    }
}
```

```java
public class DriverEx {
    public static void main(String[] args) {
        Driver driver = new Driver();

        // Vehicle 구현 객체 생성
        Bus bus = new Bus();
        driver.driver(bus);     // 구현객체대입(자동타입변환)

        Taxi taxi = new Taxi();
        driver.driver(taxi);    // 구현객체대입(자동타입변환)
    }
}
```

16-4. 인터페이스 정리

implements 와 extends 의 사용방법 정리

부모	자식	keyword	다중 상속
class	class	extends	X
interface	class	implements	O
interface	interface	extends	O

인터페이스 문법 정리

- new 를 사용한 객체 생성이 불가, 추상 메서드와 상수를 가진다.
- 선언된 메서드는 자동적으로 public abstract가 되고, 필드는 public static final이 된다.
- 인터페이스는 객체가 아니고, 객체 타입으로만 사용 가능하다.
- 복수의 인터페이스를 부모로 두는 다중 상속이 가능하다.
- 인터페이스를 부모로 두는 자식 클래스(구현 객체) 정의에 implements 를 사용한다.